TikuOS Memory Region Registry

Simple. Ubiquitous. Intelligence, Everywhere.

Ambuj Varshney
ambuj@tiku-os.org

March 8, 2026

Outline

1. Why a Region Registry?
2. Architecture & Data Structures
3. Platform Layering
4. API Reference
5. Integration
6. Usage Examples
7. File Organization
8. API Summary
9. Test Suite
10. Summary

The Problem: Wrong Buffer Placement

Without region validation:

- Arena buffer accidentally placed in FRAM
- Persistent store backed by volatile SRAM
- Two subsystems sharing the same buffer
- Stack-allocated buffer passed to arena create
- All silent, all diagnosed at 3 AM

TikuOS region registry:

- Boot-time memory map from platform HAL
- `contains()` validates buffer type
- Arena create rejects non-SRAM buffers
- Persist register rejects non-NVM buffers
- `claim()` detects overlapping buffers
- `get_type()` for diagnostics

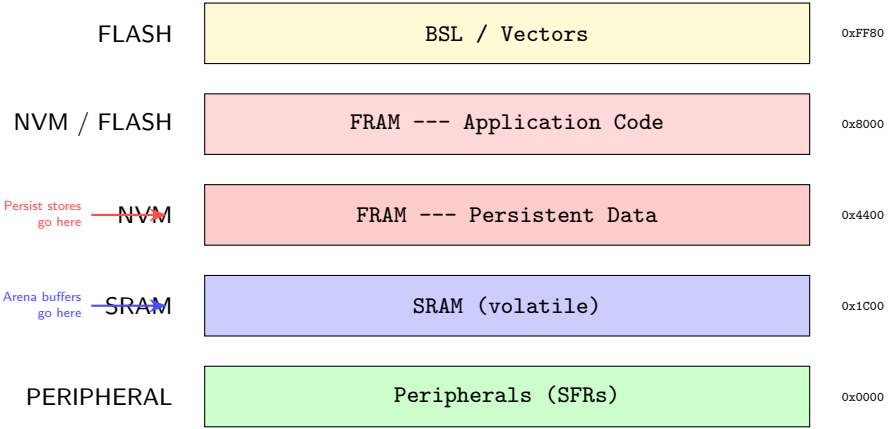
Silent Misplacement

On a microcontroller with SRAM and FRAM in the same flat address space, nothing prevents you from creating an arena in FRAM or a persistent store in SRAM. The code compiles, the system boots, and the bug manifests as unexplained data corruption — hours or days later.

Fail Fast at Init

The region registry catches buffer placement errors at `create()` / `register()` time — before the system runs. The cost is a few linear scans during boot; the hot path is untouched.

MSP430 Memory Map: Why Types Matter



Flat Address Space, Different Rules

All regions share one 16-bit address space. A pointer to `0x1D00` (SRAM) looks identical to a pointer to `0x4500` (FRAM). Only the region registry knows which is which.

Region Types & Descriptors (kernel/memory/tiku_mem.h)

Region type enum

```
typedef enum {
    TIKU_MEM_REGION_SRAM,
    TIKU_MEM_REGION_NVM,
    TIKU_MEM_REGION_PERIPHERAL,
    TIKU_MEM_REGION_FLASH
} tiku_mem_region_type_t;
```

Region descriptor

```
typedef struct tiku_mem_region {
    const uint8_t            *base;
    tiku_mem_arch_size_t      size;
    tiku_mem_region_type_t    type;
} tiku_mem_region_t;
```

Claimed region (overlap tracking)

```
typedef struct {
    const uint8_t            *base;
    tiku_mem_arch_size_t      size;
    uint8_t                   owner_id;
} tiku_mem_claimed_t;
```

Design Choices

- Struct tag `tiku_mem_region` enables forward declaration in the HAL header
- `const uint8_t *base` — region table is read-only (lives in flash)
- `owner_id` links claims to subsystems for diagnostic output

Internal State (kernel/memory/tiku_region.c)

```c
/** Pointer to platform-provided region table (lives in flash) */
static const tiku_mem_region_t *region_table;

/** Number of entries in the region table */
static tiku_mem_arch_size_t region_count;

/** Claimed region tracking array (lives in SRAM) */
static tiku_mem_claimed_t claimed[TIKU_REGION_MAX_CLAIMS];

/** Number of active claims */
static tiku_mem_arch_size_t claimed_count;
```

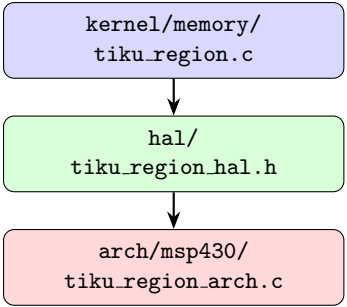
Zero-Copy Region Table

`region_table` is a *pointer* to the platform's const array — no copy is made. The table lives in flash; we just store its address and length.

Configurable Limits

`TIKU_REGION_MAX_REGIONS` (default 10) and `TIKU_REGION_MAX_CLAIMS` (default 16) are overridable at compile time for platforms with more complex memory maps.

Three-Layer Architecture



Platform-independent logic: init, contains, claim, unclaim, get_type

Forward-declares `struct tiku_mem_region`; declares `tiku_region_arch_get_table()`

Returns const table of `tiku_mem_region_t` descriptors

Same HAL Pattern as MPU and Arena

The region registry follows the same three-layer pattern: kernel logic calls a HAL-declared function, and the platform arch layer provides the implementation. Porting to a new MCU means writing one function — `tiku_region_arch_get_table()`.

HAL Interface & Forward Declaration

hal/tiku_region_hal.h

```c
#ifndef TIKU_REGION_HAL_H_
#define TIKU_REGION_HAL_H_

#include <stdint.h>
#include "hal/tiku_mem_hal.h"

/* Full definition in tiku_mem.h */
struct tiku_mem_region;

/**
 * Return the platform's memory
 * region table.
 */
const struct tiku_mem_region *
tiku_region_arch_get_table(
    tiku_mem_arch_size_t *count);

#endif /* TIKU_REGION_HAL_H_ */
```

MSP430 arch implementation

```c
static const tiku_mem_region_t
    msp430_region_table[] =
{
    { (const uint8_t *)0x1C00,
      TIKU_DEVICE_SRAM_SIZE,
      TIKU_MEM_REGION_SRAM },

    { (const uint8_t *)0x4400,
      TIKU_DEVICE_FRAM_SIZE,
      TIKU_MEM_REGION_NVM },
};

const struct tiku_mem_region *
tiku_region_arch_get_table(
    tiku_mem_arch_size_t *count)
{
    *count = 2;
    return msp430_region_table;
}
```

Forward Declaration Avoids Circular Include

tiku_region_init()

```c
tiku_mem_err_t tiku_region_init(const tiku_mem_region_t *table,
                                tiku_mem_arch_size_t count);
```

kernel/memory/tiku_region.c

```c
tiku_mem_err_t tiku_region_init(const tiku_mem_region_t *table,
                                tiku_mem_arch_size_t count) {
    /* Validate arguments */
    if (table == NULL || count == 0 || count > TIKU_REGION_MAX_REGIONS)
        return TIKU_MEM_ERR_INVALID;

    /* Check for pairwise overlaps in the platform's region table */
    for (i = 0; i < count; i++)
        for (j = i + 1; j < count; j++)
            if (ranges_overlap(table[i].base, table[i].size,
                               table[j].base, table[j].size))
                return TIKU_MEM_ERR_INVALID;

    region_table = table;   /* store pointer (no copy) */
    region_count = count;
    /* Zero the claimed regions array */
    ...
    return TIKU_MEM_OK;
}
```

tiku_region_contains()

```c
tiku_mem_err_t tiku_region_contains(const uint8_t *ptr,
                                    tiku_mem_arch_size_t size,
                                    tiku_mem_region_type_t expected_type);
```

kernel/memory/tiku_region.c

```c
tiku_mem_err_t tiku_region_contains(const uint8_t *ptr,
                                    tiku_mem_arch_size_t size,
                                    tiku_mem_region_type_t expected_type) {
    if (ptr == NULL || size == 0) return 0;

    range_start = (uintptr_t)ptr;
    range_end   = range_start + (uintptr_t)size;
    if (range_end < range_start) return 0;  /* overflow check */

    for (i = 0; i < region_count; i++) {
        if (region_table[i].type != expected_type) continue;
        reg_start = (uintptr_t)region_table[i].base;
        reg_end   = reg_start + (uintptr_t)region_table[i].size;
        if (range_start >= reg_start && range_end <= reg_end) return 1;
    }
    return 0;
}
```

tiku_region_claim() / tiku_region_unclaim()

Claim — register ownership

```c
tiku_mem_err_t tiku_region_claim(
    const uint8_t *ptr,
    tiku_mem_arch_size_t size,
    uint8_t owner_id)
{
    if (ptr == NULL || size == 0)
        return TIKU_MEM_ERR_INVALID;

    /* Must be in a declared region */
    if (!region_is_known(ptr, size))
        return TIKU_MEM_ERR_INVALID;

    /* Check overlap with claims */
    for (i = 0; i < MAX_CLAIMS; i++)
        if (claimed[i].size != 0 &&
            ranges_overlap(...))
            return TIKU_MEM_ERR_INVALID;

    /* Store in first empty slot */
    claimed[i] = { ptr, size, id };
    return TIKU_MEM_OK;
}
```

Unclaim — release ownership

```c
tiku_mem_err_t tiku_region_unclaim(
    const uint8_t *ptr)
{
    if (ptr == NULL)
        return TIKU_MEM_ERR_NOT_FOUND;

    for (i = 0; i < MAX_CLAIMS; i++) {
        if (claimed[i].size != 0 &&
            (uintptr_t)claimed[i].base
            == (uintptr_t)ptr) {
            claimed[i] = { 0 };
            claimed_count--;
            return TIKU_MEM_OK;
        }
    }
    return TIKU_MEM_ERR_NOT_FOUND;
}
```

Three Validations

`claim()` verifies:

tiku_region_get_type()

```c
tiku_mem_err_t tiku_region_get_type(const uint8_t *ptr,
                                    tiku_mem_region_type_t *out_type);
```

kernel/memory/tiku_region.c

```c
tiku_mem_err_t tiku_region_get_type(const uint8_t *ptr,
                                    tiku_mem_region_type_t *out_type) {
    if (ptr == NULL || out_type == NULL) return TIKU_MEM_ERR_NOT_FOUND;

    addr = (uintptr_t)ptr;
    for (i = 0; i < region_count; i++) {
        reg_start = (uintptr_t)region_table[i].base;
        reg_end   = reg_start + (uintptr_t)region_table[i].size;
        if (addr >= reg_start && addr < reg_end) {
            *out_type = region_table[i].type;
            return TIKU_MEM_OK;
        }
    }
    return TIKU_MEM_ERR_NOT_FOUND;
}
```

Diagnostic Utility

get_type() looks up a single address and returns which region type it belongs to. Useful for debug

Private Helper: ranges_overlap()

kernel/memory/tiku_region.c

```
static int ranges_overlap(const uint8_t *a_base, tiku_mem_arch_size_t a_size,
                          const uint8_t *b_base, tiku_mem_arch_size_t b_size)
{
    uintptr_t a_start = (uintptr_t)a_base;
    uintptr_t a_end   = a_start + (uintptr_t)a_size;
    uintptr_t b_start = (uintptr_t)b_base;
    uintptr_t b_end   = b_start + (uintptr_t)b_size;

    return (a_start < b_end) && (b_start < a_end);
}
```

Classic Interval Overlap Test

Two half-open intervals $[a, a{+}s)$ and $[b, b{+}s)$ overlap if and only if $a < b_\text{end}$ AND $b < a_\text{end}$. This is the standard algorithm from computational geometry.

Used Twice

1. **Init:** Rejects overlapping platform regions
2. **Claim:** Detects conflicting buffer assignments

Adjacent ranges (`a_end == b_start`) do **not** overlap — this is correct because half-open intervals exclude the end point.

Integration into Existing Subsystems

tiku_arena_create() — added check

```
tiku_mem_err_t tiku_arena_create(
    tiku_arena_t *arena,
    uint8_t *buf,
    tiku_mem_arch_size_t size,
    uint8_t id)
{
    /* NULL checks ... */

    /* NEW: buffer must be in SRAM */
    if (!tiku_region_contains(buf, size,
            TIKU_MEM_REGION_SRAM))
        return TIKU_MEM_ERR_INVALID;

    /* NEW: claim for overlap detect */
    tiku_region_claim(buf, size, id);

    /* ... existing init code ... */
}
```

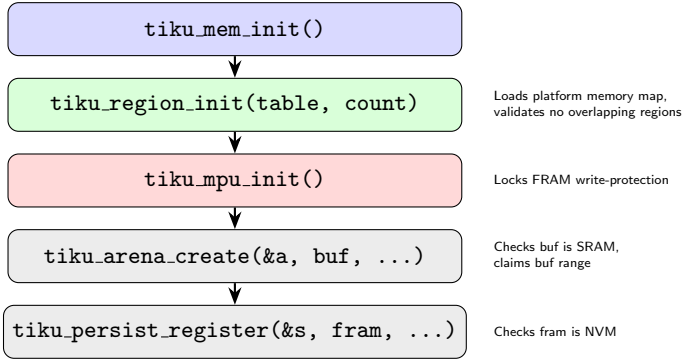
tiku_persist_register() — added check

```
tiku_mem_err_t tiku_persist_register(
    tiku_persist_store_t *store,
    uint8_t *fram_buf,
    tiku_mem_arch_size_t capacity,
    ...)
{
    /* NULL checks ... */

    /* NEW: buffer must be in NVM */
    if (!tiku_region_contains(
            fram_buf, capacity,
            TIKU_MEM_REGION_NVM))
        return TIKU_MEM_ERR_INVALID;

    /* ... existing init code ... */
}
```

Two Lines per Subsystem

Each allocator gains a `contains()` guard and an optional `claim()`. Minimal code change.

Boot Sequence with Region Registry



Order Matters

Region init runs first so that arena and persist can validate their buffers. MPU init runs second so that FRAM is write-protected before any application code. Arena and persist run last, using both the region registry (for type checking) and the MPU (for write protection).

Example 1: The Bug Region Registry Catches

Bug: arena in FRAM

```
/* Developer places arena buffer in
   FRAM by mistake (wrong section) */
static uint8_t __attribute__((
    section(".persistent")))
    scratch[128] = {0};

static tiku_arena_t arena;

void init(void) {
    /* Without region check:
       silently creates arena in FRAM.
       Every alloc writes to NVM --
       wears out FRAM, wastes cycles,
       data persists unexpectedly. */
    tiku_arena_create(&arena,
        scratch, 128, 1);
}
```

With region registry

```
void init(void) {
    tiku_mem_err_t err;
    err = tiku_arena_create(&arena,
        scratch, 128, 1);

    /* tiku_region_contains(scratch,
       128, TIKU_MEM_REGION_SRAM)
       returns 0 --- scratch is in
       NVM, not SRAM!

       arena_create returns
       TIKU_MEM_ERR_INVALID.

       Bug caught at init time. */

    if (err != TIKU_MEM_OK)
        fault_handler("bad arena buf");
}
```

Caught Immediately

Instead of silent FRAM wear, the error is reported

Example 2: Overlapping Buffer Detection

```c
/* Two subsystems accidentally share the same SRAM range */
static uint8_t shared_buf[256];

static tiku_arena_t net_arena;
static tiku_arena_t sensor_arena;

void init(void) {
    tiku_mem_err_t err;

    /* First arena claims [shared_buf, shared_buf + 128) */
    err = tiku_arena_create(&net_arena, shared_buf, 128, 1);
    assert(err == TIKU_MEM_OK);  /* succeeds */

    /* Second arena tries [shared_buf + 64, shared_buf + 192) -- overlaps! */
    err = tiku_arena_create(&sensor_arena, shared_buf + 64, 128, 2);
    /* tiku_region_claim detects overlap with net_arena's claim */
    /* Returns TIKU_MEM_ERR_INVALID --- overlap caught at boot */
    assert(err == TIKU_MEM_ERR_INVALID);  /* bug caught! */
}
```

Why This Matters on MCUs

On a desktop OS, overlapping allocations cause an immediate segfault. On a bare-metal MCU with no MMU, two subsystems silently corrupt each other's data. The claim table provides the same protection

Example 3: Runtime Diagnostics with get_type()

```c
/* Debug helper: print which memory type a pointer belongs to */
void debug_print_ptr_info(const char *name, const uint8_t *ptr) {
    tiku_mem_region_type_t type;
    tiku_mem_err_t err;

    err = tiku_region_get_type(ptr, &type);
    if (err != TIKU_MEM_OK) {
        TIKU_PRINTF("%s @ %p: UNKNOWN MEMORY\n", name, (void *)ptr);
        return;
    }

    const char *type_names[] = {"SRAM", "NVM", "PERIPHERAL", "FLASH"};
    TIKU_PRINTF("%s @ %p: %s\n", name, (void *)ptr, type_names[type]);
}

/* Output:
   arena_buf @ 0x1C00: SRAM
   persist_buf @ 0x4400: NVM
   stack_var @ 0x23F0: SRAM        */
```

Debugging Aid

When tracking down memory corruption, `get_type()` immediately tells you whether a pointer is in the right region — without consulting a linker map file.

Example 4: Dynamic Buffer Registration

```c
/* Module initializes at runtime, not at compile time.
   Region registry validates the buffer before use. */

tiku_mem_err_t radio_init(uint8_t *rx_buf, tiku_mem_arch_size_t rx_size) {
    /* Validate: caller must provide SRAM buffer */
    if (!tiku_region_contains(rx_buf, rx_size, TIKU_MEM_REGION_SRAM)) {
        return TIKU_MEM_ERR_INVALID;  /* wrong memory type */
    }

    /* Claim: prevent other subsystems from using same range */
    tiku_mem_err_t err = tiku_region_claim(rx_buf, rx_size, RADIO_ID);
    if (err != TIKU_MEM_OK) {
        return err;  /* overlap or table full */
    }

    /* Safe to use rx_buf as radio receive buffer */
    radio_hw_set_rx_buffer(rx_buf, rx_size);
    return TIKU_MEM_OK;
}
```

Beyond Arenas and Persist

Any subsystem can use `contains()` and `claim()` — not just arenas and persistent stores. DMA buffers, radio buffers, and display framebuffers all benefit from type validation.

Module File Organization

Layer	File	Role
Kernel (portable)	`kernel/memory/tiku_mem.h` `kernel/memory/tiku_region.c`	Types, enums, function prototypes Registry logic: init, contains, claim
HAL	`hal/tiku_region_hal.h`	Forward declaration, HAL accessor
Arch (MSP430)	`arch/msp430/tiku_region_arch.c`	Platform memory map table

Tests	`tests/memory/test_mem_region.c`
	`tests/memory/test_mem_common.c`
	`tests/tiku_test_config.h`
		12 region test cases
		Host stubs + test main
		Per-test enable flags

Porting to a New Architecture

Implement one function: `tiku_region_arch_get_table()` — return a const array of `tiku_mem_region_t` describing the platform's SRAM, NVM, and peripheral address ranges. The kernel region logic and all tests remain untouched.

API at a Glance

Function	Returns	Complexity	Description
`tiku_region_init`	`err_t`	$O(n^2)$	Load table, validate no overlaps
`tiku_region_contains`	`int`	$O(n)$	Check range vs. type
`tiku_region_claim`	`err_t`	$O(c)$	Register ownership, detect overlap
`tiku_region_unclaim`	`err_t`	$O(c)$	Release ownership by base ptr
`tiku_region_get_type`	`err_t`	$O(n)$	Look up region type for address

n = number of platform regions (typically 2–4), c = max claims (default 16). All scans are linear over small arrays — no hot-path impact.

Safety Guarantees

- Type validation at init time
- Overlap detection for all claims
- Pointer overflow protection (16-bit)
- `uintptr_t` for defined behavior
- NULL and zero-size rejection

Design Choices

- Zero-copy region table (flash pointer)
- No dynamic allocation anywhere
- Boot-time only — zero hot-path cost
- Configurable table sizes
- Same HAL pattern as MPU and arena

Test Configuration

Host-native (no hardware):

```
gcc -std=c99 -Wall -Wextra -I. \
  tests/memory/test_mem_common.c \
  tests/memory/test_mem_region.c \
  kernel/memory/tiku_region.c \
  kernel/memory/tiku_mem.c \
  -o test_region && ./test_region
```

Host mode maps two static pools — `test_sram_pool` (SRAM) and `test_nvm_pool` (NVM) — via a stub `tiku_region_arch_get_table()`.

Host region table stub

```
static const tiku_mem_region_t
    test_region_table[] = {
    { test_sram_pool,
      sizeof(test_sram_pool),
      TIKU_MEM_REGION_SRAM }, 
    { test_nvm_pool,
      sizeof(test_nvm_pool),
      TIKU_MEM_REGION_NVM  },
};

const struct tiku_mem_region *
tiku_region_arch_get_table(
    tiku_mem_arch_size_t *count)
{
    *count = 2;
    return test_region_table;
}
```

Test Cases Overview

#	Test Flag	What It Verifies
46	`TEST_REGION_INIT`	Valid table accepted; count ≥ 2
47	`TEST_REGION_INIT_INVALID`	NULL table, zero count, overlapping regions rejected
48	`TEST_REGION_CONTAINS`	SRAM→SRAM passes; NVM→NVM passes; NULL/zero rejected
49	`TEST_REGION_WRONG_TYPE`	SRAM→NVM fails; NVM→SRAM fails; SRAM→PERIPHERAL fails
50	`TEST_REGION_BOUNDARY`	Exact region size passes; +1 byte fails; single byte at edges
51	`TEST_REGION_OVERFLOW`	Pointer near `0xFFFF` with wrapping size rejected
52	`TEST_REGION_CLAIM`	Claim/unclaim cycle; re-claim after unclaim; double unclaim
53	`TEST_REGION_CLAIM_OVERLAP`	Overlapping and duplicate claims rejected; adjacent OK
54	`TEST_REGION_CLAIM_UNKNOWN`	Claim outside all regions rejected; NULL/zero rejected
55	`TEST_REGION_CLAIM_FULL`	Fill all claim slots; next claim returns `ERR_FULL`
56	`TEST_REGION_GET_TYPE`	SRAM and NVM addresses return correct type; boundary addresses

Test Deep Dive: Overlap Detection

test_mem_region.c — Claim overlap detection

```c
void test_region_claim_overlap(void) {
    tiku_mem_err_t err;
    test_region_reinit();

    /* Claim [0, 64) */
    err = tiku_region_claim(test_sram_pool, 64, 1);
    TEST_ASSERT(err == TIKU_MEM_OK, "first claim succeeds");

    /* Overlapping claim [32, 96) -- should fail */
    err = tiku_region_claim(test_sram_pool + 32, 64, 2);
    TEST_ASSERT(err == TIKU_MEM_ERR_INVALID, "overlapping claim rejected");

    /* Exact same range -- also fails */
    err = tiku_region_claim(test_sram_pool, 64, 3);
    TEST_ASSERT(err == TIKU_MEM_ERR_INVALID, "duplicate claim rejected");

    /* Non-overlapping [128, 192) -- succeeds */
    err = tiku_region_claim(test_sram_pool + 128, 64, 4);
    TEST_ASSERT(err == TIKU_MEM_OK, "non-overlapping claim succeeds");

    /* Adjacent [64, 128) -- no overlap, succeeds */
    err = tiku_region_claim(test_sram_pool + 64, 64, 5);
    TEST_ASSERT(err == TIKU_MEM_OK, "adjacent claim succeeds");
}
```

Test Deep Dive: Boundary & Overflow

Boundary conditions

```c
void test_region_contains_boundary(void)
{
    int result;
    test_region_reinit();

    /* Exact region boundary -- OK */
    result = tiku_region_contains(
        test_sram_pool,
        TEST_SRAM_POOL_SIZE,
        TIKU_MEM_REGION_SRAM);
    TEST_ASSERT(result == 1);

    /* One byte past -- fail */
    result = tiku_region_contains(
        test_sram_pool,
        TEST_SRAM_POOL_SIZE + 1,
        TIKU_MEM_REGION_SRAM);
    TEST_ASSERT(result == 0);

    /* Single byte at last valid pos */
    result = tiku_region_contains(
        test_sram_pool +
            TEST_SRAM_POOL_SIZE - 1,
        1, TIKU_MEM_REGION_SRAM);
```

Pointer overflow protection

```c
void test_region_contains_overflow(void)
{
    uintptr_t near_max;
    const uint8_t *wrap_ptr;
    int result;
    test_region_reinit();

    /* Craft pointer near top of
       address space so ptr + size
       wraps around to zero */
    near_max =
        ~(uintptr_t)0 - 5U;
    wrap_ptr =
        (const uint8_t *)near_max;

    result = tiku_region_contains(
        wrap_ptr, 20,
        TIKU_MEM_REGION_SRAM);
    TEST_ASSERT(result == 0,
        "wrapping range rejected");
}
```

Summary

Region Registry Design

- Boot-time memory type validation
- SRAM vs NVM vs Peripheral vs Flash
- Overlap detection via claim table
- `uintptr_t` for defined pointer comparison
- 16-bit overflow protection
- Zero-copy platform table (flash pointer)
- No dynamic allocation
- Configurable table sizes
- Platform-independent kernel code

Test Coverage

- 12 comprehensive test cases
- Runs on-target or host-native
- Per-test enable flags
- Covers: init, contains, wrong type, boundary, overflow, claim/unclaim, overlap, unknown, full, get_type

Integrated Into All Allocators

Arena `create()` checks SRAM. Persist `register()` checks NVM. Both call `claim()` for overlap detection. All validation happens at init time — zero cost on the hot path.

Thank You

TikuOS Memory Region Registry

Boot-Time Memory Validation for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
API Header:	`kernel/memory/tiku_mem.h`
Implementation:	`kernel/memory/tiku_region.c`
HAL:	`hal/tiku_region_hal.h`
Tests:	`tests/memory/test_mem_region.c`